AttendEase

> Multi-tenant SaaS Employee Attendance Management System

What Is AttendEase?

AttendEase is a cloud-based attendance management platform that eliminates buddy punching and GPS spoofing through a combination of facial recognition and polygon geo-fencing. Employees mark attendance via a mobile app using a selfie and their GPS location. The system verifies identity via AWS Rekognition and confirms physical presence inside the office boundary before recording attendance.

Core Features

- **Selfie + GPS Check-In** — Employee takes a selfie and the app reads GPS at the moment of check-in
- **Face Recognition** — AWS Rekognition verifies employee identity, prevents buddy punching
- **Polygon Geo-Fence** — Admin draws exact office boundary on a map once; Ray Casting algorithm checks every check-in at zero API cost
- **Time Policies** — Auto-calculates Late / Half Day / Overtime per employee shift
- **Leave Management** — Apply, approve, reject leaves with half-day support
- **Regularisation** — Employees can request correction of missed or wrong attendance
- **Push Notifications** — FCM-powered alerts for checkout reminders, leave approvals, late marks
- **Admin Dashboard** — Web app to manage employees, branches, shifts, leaves, reports
- **Super Admin Portal** — Separate portal to manage all client organisations, billing, platform monitoring

---

Tech Stack

Frontend

App	Technology	Purpose
Employee Mobile App	React Native	iOS + Android attendance app
Admin Dashboard	React.js	Organisation-level management
Super Admin Portal	React.js (separate build)	Platform-level management

Backend

Layer	Technology	Purpose
Runtime	Node.js	Server-side JavaScript runtime
Framework	Express.js	REST API framework
Auth	JWT + bcrypt	Token-based auth, password hashing
Real-time	Socket.io	Live notifications, attendance board
Job Queue	Bull Queue (Redis-backed)	Delayed jobs — checkout reminders, auto-absent
Cache	Redis	Session cache, rate limiting, cooldowns

Database

Technology	Purpose
PostgreSQL (AWS RDS)	Primary relational database
Redis	Cache + Bull Queue job storage

Cloud & Infrastructure

Service	Technology	Purpose
Server	AWS EC2	Application hosting
Database	AWS RDS (PostgreSQL)	Managed database hosting
Face Recognition	AWS Rekognition	Cloud-level identity verification
Image Storage	Cloudinary	Selfie storage, face image management
Push Notifications	Firebase FCM	Push delivery to Android + iOS
Location (Admin)	Google Maps API	Polygon geo-fence drawing in admin dashboard only

On-Device (Mobile)

Technology	Purpose
ML Kit (Android) / Vision Framework (iOS)	On-device face quality pre-check before cloud call

Technology	Purpose
TensorFlow.js on EC2	Local face embedding comparison — handles 90% of check-ins without AWS

Architecture

Deployment Model

- **Monolith** to start — single Node.js application handles all services
- Split into microservices only at **50,000+ employees**

Multi-Tenancy

- Shared PostgreSQL database
- Every table has `org_id` foreign key — strict tenant isolation at query level
- One missed `org_id` filter = data leak — enforced via middleware on every request

System Layers

```
Client Layer    →  React Native App, React Admin, React Super Admin
API Gateway     →  Express.js REST API, JWT auth middleware
Service Layer   →  AttendanceService, FaceService, NotificationService, LeaveService
Queue / Cache   →  Bull Queue (delayed jobs), Redis (cache, rate limiting)
Data Layer      →  PostgreSQL (RDS), Cloudinary (images)
```

Domain Separation

Domain	URL	Auth
Employee App	Mobile app	JWT role: employee
Admin Dashboard	`app.attendease.com`	JWT role: admin
Super Admin Portal	`admin.attendease.com`	JWT role: superadmin, org_id: null

Geo-Fencing

Primary — Polygon Geo-Fence

- Admin draws exact building boundary **once** on Google Maps in the admin dashboard

- Stored as `JSONB` array of lat/lng coordinates in `branches.geo_fence_polygons`
- **Ray Casting (Point-in-Polygon)** algorithm runs per check-in in Node.js — pure math, zero API cost
- Multiple polygons supported for campuses

Fallback — Haversine (Circle)

- Used only when no polygon is drawn
- Calculates straight-line distance between two GPS coordinates
- Runs entirely in Node.js — zero API calls

Ray Casting Rule

- Fire an imaginary ray from employee GPS point going east
- Count how many times it crosses the polygon boundary
- **Odd crossings = INSIDE ✅ | Even crossings = OUTSIDE ❌**

GPS Accuracy Thresholds

Accuracy	Action
≤ 50m	Proceed normally
50–100m	Allow but flag in DB
> 100m	Ask employee to retry near a window

Face Recognition

7-Layer Optimization Stack

Layer	Technology	Cost	Handles
1. On-device quality check	ML Kit / Vision	$0	Blur, dark, no face, multiple faces
2. Local embedding compare	TensorFlow.js on EC2	$0	High-confidence matches (~90% of check-ins)
3. Trust scoring	App logic	$0	Relax threshold for long-tenure employees
4. Deduplication	Redis (5-min window)	$0	Double taps, app bugs
5. Session cache	Redis (10-min window)	$0	Re-verification in same session
6. AWS Rekognition	Rekognition API	$0.001/call	Borderline cases only (~8%)

Confidence Thresholds

Employee Type	Threshold
New (< 30 days)	0.88 — strict
Default	0.84
Trusted (200+ check-ins)	0.80 — relaxed
Flagged (suspicious history)	0.95 — very strict

Result

- Without optimization: ~500,000 AWS calls/month = **$500/month**
- With optimization: ~35,000 AWS calls/month = **$35/month**
- **93% cost reduction ($465/month saved)**

Location Strategy

Permission Model

- **"While Using App" ONLY** — never "Always On"
- "Always On" is illegal under India's **DPDP Act 2023** and gets apps rejected on App Store
- Location read ~30 seconds at check-in + ~30 seconds at check-out = **0.07% of the day**

GPS Mock Detection — 4 Layers

Layer	Where	Signal
OS Flag	App (Android only)	`position.mocked = true`
Developer Mode	App	`isLocationMocked() = true`
Velocity Check	Backend	Speed > 300 km/h between two check-ins
GPS Pattern	Backend	Suspiciously perfect accuracy always ±0–2m

- iOS has **no** `position.mocked` flag — velocity check is the primary anti-spoof for iOS
- Never tell the employee which layer caught them
- Show only: *"Location verification failed. Please disable location simulation apps."*

WFH Employees

- `branch.is_remote = true` → geo-fence check skipped
- Face recognition still runs
- Location still stored for audit purposes but not validated

Notification System

Delivery Stack

Layer	Tool	Job
Scheduling	Bull Queue	When to send (delayed jobs)
Delivery	Firebase FCM	How to reach the phone
Real-time	Socket.io	Live badge update when app is open
History	`notifications` table	In-app bell, read/unread state

Notification Triggers

Trigger	Queue Needed?	FCM Type
Forgot checkout	✅ Delayed (Bull Queue)	Single
Leave approved / rejected	❌ Immediate	Single
Late / absent marked	❌ Immediate	Single
Regularisation result	❌ Immediate	Single
Admin broadcast	✅ Bull Queue	Multicast (500 tokens/call)

Cost

Firebase FCM is completely free with no limits. Entire notification system costs **$0/month**.

Database — 12 Tables

Table	Key Design
organisations	Root table. slug unique. settings JSONB. plan enum.
branches	geo_fence_polygons JSONB. is_remote BOOLEAN. Check-in window times.
departments	Self-referencing parent_id for hierarchy.
shifts	crosses_midnight BOOLEAN for night shifts. work_days INTEGER[].
employees	face_embedding_id = AWS Rekognition FaceId. leave_balance JSONB.
attendance	date DATE + check_in_time TIMESTAMPTZ separate for fast queries. Partition by month at >1M rows.
leave_requests	days_count DECIMAL(4,1) for half-day. GIST exclude prevents overlapping leaves.
regularisations	Links to attendance_id. One pending per attendance at a time.
holidays	branch_id = NULL = org-wide. is_recurring for annual repeats.
notifications	action_url deep links. Partial index on is_read = FALSE.
device_tokens	UNIQUE(emp_id, fcm_token). Partial index on is_active = TRUE.
audit_logs	BIGSERIAL (not UUID) for guaranteed ordering. Partition monthly.

Schema Ground Rules

1. UUID primary keys on all tables except audit_logs (BIGSERIAL)
2. org_id FK on every single table — multi-tenancy
3. created_at + updated_at on every table — trigger auto-updates updated_at
4. Soft deletes via deleted_at — never hard delete employees or orgs
5. settings JSONB for flexible config — avoids migrations for new features

---

Infrastructure Requirements

Development (Local)

Requirement	Version
Node.js	v18+ LTS

Requirement	Version
PostgreSQL	v15+
Redis	v7+
React Native CLI	Latest
Android Studio	Latest (for Android emulator)
Xcode	Latest (for iOS simulator, macOS only)

Third-Party Accounts Required

Service	Purpose	Cost
AWS Account	EC2, RDS, Rekognition	Pay as you go
Firebase Project	FCM push notifications, ML Kit	Free
Google Cloud	Maps API (admin dashboard only)	Pay per use (one-time polygon draw)
Cloudinary	Selfie image storage	Free tier to start
Apple Developer Account	iOS App Store distribution + APNs	$99/year
Google Play Console	Android App Store distribution	$25 one-time

Environment Variables Required

```bash
```

```
# Database
DATABASE_URL=postgresql://user:password@rds-endpoint:5432/attendease
REDIS_URL=redis://localhost:6379

# Auth
JWT_SECRET=your_jwt_secret
JWT_REFRESH_SECRET=your_refresh_secret
JWT_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d

# AWS
AWS_ACCESS_KEY_ID=your_access_key
AWS_SECRET_ACCESS_KEY=your_secret_key
AWS_REGION=ap-south-1
AWS_REKOGNITION_COLLECTION_ID=attendease-faces

# Firebase
FIREBASE_SERVICE_ACCOUNT_JSON={"type":"service_account",...}

# Google Maps
GOOGLE_MAPS_API_KEY=your_maps_key

# Cloudinary
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

# App
NODE_ENV=development
PORT=3000
FRONTEND_URL=http://localhost:5173
```

Production Cost Estimates

At 10,000 Employees

Service	Monthly Cost
AWS EC2 (t3.medium)	~$30
AWS RDS PostgreSQL (db.t3.medium)	~$60
Redis (ElastiCache t3.micro)	~$15

Service	Monthly Cost
AWS Rekognition (optimized ~35K calls)	~$35
Cloudinary (storage + bandwidth)	~$25
Google Maps API (polygon draw only)	~$5
Firebase FCM	$0
Firebase ML Kit	$0
Total	**~$170–$200/month**

Scaling Thresholds

Employees	Monthly Infra Cost
0 – 3,000	~$290
3,000 – 10,000	~$381
10,000 – 50,000	~$600
50,000+	~$1,200+ (consider microservices)

Revenue Projection

Metric	Value
Pricing	₹50 per employee per month
Revenue at 10,000 employees	₹5,00,000/month (~$6,000)
Infra cost	~$381/month
Gross Margin	**~93%**

Monorepo Folder Structure (Planned)

```
attendease/
├── backend/                # Node.js + Express API
│   ├── src/
│   │   ├── routes/
│   │   ├── services/
```

```
│   │   ├── queues/
│   │   ├── workers/
│   │   ├── middleware/
│   │   └── utils/
│   └── package.json
│
├── admin-dashboard/        # React.js — org-level admin
│   └── package.json
│
├── super-admin/            # React.js — platform-level admin
│   └── package.json
│
├── mobile-app/             # React Native — employee app
│   └── package.json
│
├── shared/                 # Shared types, constants, utils
│   └── package.json
│
└── package.json            # Root monorepo config
```

Known Pain Points

Development

- Face Recognition AWS learning curve — collection management, FaceId vs raw embedding
- Offline sync conflict resolution — Bull Queue auto-absent vs offline check-in arriving late
- React Native GPS accuracy indoors — `watchPosition` pre-warming required
- JWT refresh on mobile when offline for days
- Multi-tenancy `org_id` enforcement — one missed filter = data leak

Production

- Face recognition in poor light or with masks — configurable confidence threshold needed
- GPS spoofing — 4-layer mock detection + velocity check required
- Bull Queue reliability — dead-letter queue + admin alerting mandatory
- Cloudinary storage cost at scale — monitor, enforce 90-day retention
- Timezone handling — all time calculations must use `organisations.timezone`
- Audit log table growth — partition by month from day one, not later

---

Topics Completed in Planning

- [x] App idea, features, tech stack, high-level design, check-in flow, cost estimates
- [x] Haversine formula — explanation + Node.js implementation
- [x] Polygon geo-fence — Ray Casting algorithm + code + interactive explainer
- [x] UI layouts — Mobile (6 screens) + Admin (9 screens) + Super Admin (8 screens)
- [x] ER diagram — 12 tables with all relationships
- [x] Super Admin portal — full feature set + 8 screens
- [x] Detailed DB schema — all 12 tables, indexes, constraints, triggers
- [x] Employee location strategy — permissions, pipeline, mock detection, WFH, Android/iOS diff
- [x] Face recognition cost optimization — 7-layer strategy, 93% cost reduction
- [x] Notification system — FCM, Bull Queue, deep linking, offline handling, rate limiting, badge count
- [x] Technical documentation — full Word document

Topics Remaining

- [ ] Folder Structure — monorepo layout for all 4 apps
- [ ] API Endpoint Design — REST routes, request/response, error codes, versioning
- [ ] Face Recognition Flow (detailed) — enroll, match, collection management
- [ ] JWT + Auth Flow — access/refresh tokens, offline expiry, impersonation
- [ ] Offline Sync Logic — local storage, sync on reconnect, conflict resolution
- [ ] Multi-Tenancy in PostgreSQL — RLS vs org_id middleware strategy
- [ ] Payroll Webhook / HR Integration — Darwinbox, Keka, GreytHR
- [ ] Low Level Design — Bull Queue job designs, auto-absent, OT/late/half-day engine
- [ ] Sprint Planning — Phase 1 MVP, Phase 2, Phase 3, team structure, timeline

---

Last updated: March 2026